

3

RELATION MODEL AND SQL

3.1 Procedural Query Language and Non-procedural Query Language

Procedural Query Language	Non-procedural Query Language
Formulation of how to access data from the database table and what data required to retrieve from DB tables. “Relational Algebra”	Formulation of what data retrieve from DB tables. “Relational Calculus”

3.2 Relational Algebra

(Always generate distinct tuples)

Relational algebra refers to a procedural query language that takes relation instances as input and returns relation instances as output.

3.2.1 Basic operators

- π : Projection operator
- σ : Selection operator
- $\times$: Cross-product operator
- $\cup$: Union
- $-$: Set difference
- ρ : Rename operator

3.2.2 Derived operators

- $\cap$: Intersection {using “ $_$ ”}
- $\bowtie$: Join {using X, σ }
- $/$ or $\div$: Division {using $\pi, x, -$ }

I. π : Projection

- $\pi_{\text{attribute_name}}(R)$: It is used to project required attribute from relation R.
- $\sigma_{\text{Condition}(P)}(R)$: It is used to select records from relation R, those satisfied the condition (P).



Example:

R	A	B	C
	8	4	5
	2	4	5
	7	4	6
	3	5	5

 $\left. \vphantom{\begin{matrix} R \\ \pi_{B,C}(R) \\ \sigma_{A>5}(R) \end{matrix}} \right\}$

$\pi_{B,C}(R) :$	B	C
	4	5
	4	6
	5	5

$\sigma_{A>5}(R) :$	A	B	C
	8	4	5
	7	4	6

II. Cross product ($\times$):

- $R \times S$: It result all attributes of R followed by all attributes of S, and each record of R paired with every record of S.
- Degree ($R \times S$) = Degree (R) + Degree (S)
- $|R \times S| = |R| \times |S|$

Note:

- Relation R with n tuples and
- Relation S with 0 tuples then
number of tuples in $R \times S = 0$ tuples

3.3 Join ($\bowtie$)

I. Natural join ($\bowtie$)

$$R \bowtie S \equiv \pi_{\text{distinct attributes}}(\sigma_{\text{equality between common attributes of R and S}}(R \times S))$$

Example:

- T_1 (ABC) and T_2 (BCDE)

$$\therefore T_1 \bowtie T_2 = \pi_{ABCDE} \left(\begin{matrix} \sigma_{T_1.B=T_2.B} (T_1 \times T_2) \\ \cap T_1.C=T_2.C \end{matrix} \right)$$

- T_1 (AB) and T_2 (CD)

$$\therefore T_1 \bowtie T_2 \equiv T_1 \times T_2 = \pi_{ABCD} (T_1 \times T_2)$$

Note:

Natural join equal to cross-product if join condition is empty.

II. Conditional Join ($\bowtie_c$)

- $R \bowtie_c S \equiv \sigma_c (R \times S)$

III Outer Joins:

(a) LEFT OUTER JOIN

$R \bowtie S$: It produces

$(R \bowtie S) \cup \{\text{Records of R those are failed join condition with remaining attributes null}\}$

(b) RIGHT OUTER JOIN ($\bowtie_r$)

$R \bowtie_r S$: It produces]

$(R \bowtie S) \cup \{\text{Records of S those are failed join condition with remaining attributes null}\}$

**(C) FULL OUTER JOIN ($\bowtie$)**

$$R \bowtie S = (R \bowtie S) \cup (R \bowtie S)$$

3.4 Rename operator (ρ)

- It is used to rename table name and attribute names for query processing.

Example:

(I) Stud (Sid, Sname, age)

$\rho(\text{Temp, Stud}) : \text{Temp (Sid, Sname, age)}$

(II) $\rho_{I,N,A}(\text{Stud}) : \text{Stud (I, N, A)}$

All attributes renaming

(III) $\rho_{\substack{\text{Sid} \rightarrow I \\ \text{age} \rightarrow A}}(\text{Stud}) : \text{Stud (I, Sname, A)}$

$\text{age} \rightarrow A$

Some attribute renaming

Example:

Retrieve eicls of female employees whoose salary more than every male employee?

Result :

$$\pi_{eid} \left(\sigma_{(gen=Female)}^{(Emp)} \right) - \pi_{eid} \left(E \bowtie \delta_{I,S,G} (emp) \right)$$

$\left[\begin{array}{c} gen = female \\ \cap \\ G = male \\ \cap \\ Sal \leq S \end{array} \right]$

3.5 Division

- It is used to retrieve attribute value of R which has paired with every attribute value of other relation S.
 $\pi_{AB}(R)/\pi_B(S)$: It will retrieves values of attribute 'A' from R for which there must be pairing 'B' value for every 'B' of S.

Expansion of '/' by using basic operator

- Example: Retrieve sid's who enrolled every course.

Result:

$$\pi_{sidcid}(\text{Enroll}) / \pi_{cid}(\text{Course})$$

Step 1: Sid's not enrolled every course of course relation.

(Sid's enrolled proper subset of course)

$$\pi_{sid}((\pi_{sid}(\text{Enroll}) \times \pi_{cid}(\text{course})) - \pi_{sidcid}(\text{Enroll}))$$



Step 2:

$$[\text{sid's enrolled every course}] = [\text{sid's enrolled some course}] - [\text{sid's not enrolled every course}]$$

$$\therefore \pi_{\text{sidcid}}(E) / \pi_{\text{cid}}(C) = \pi_{\text{sid}}(E) - \pi_{\text{sid}}((\pi_{\text{sid}}(E) \times \pi_{\text{cid}}(C) - \pi_{\text{sidcid}}(E))$$

3.6 Set operator

U: Union operator

– : Except minus

$\cap$: Intersection operator

- To apply set operations relations must be union compatible.
- R and S relations union compatible
- If and only if-
 - (i) Arity of R equal to Arity of S and
 - (ii) Domain of attributes of R must be same as domain of attributes of S respectively.

Example 1:

$$\pi_{\text{Sid Sname}}(\dots\dots\dots) \cap \pi_{\text{Sid}}(\dots\dots\dots)$$

{Arity not same so, set operation not allowed}

Example 2:

$$\pi_{\text{Sid Sname}}(\dots\dots\dots) \cap \pi_{\text{Sid marks}}(\dots\dots\dots)$$

{Arity same but Sname domain is different from marks so, not allowed}

Example 3:

$$\pi_{\text{Sid Sname}}(\dots\dots\dots) \cap \pi_{\text{Stud ID, Stud name}}(\dots\dots\dots)$$

{Arity and domains are same so, allowed for set operation}

- 1. Set operation on relation:

R	A	S	B
	2		2
	2		2
	2		4
	3		